

מדריך הקמת מערכת NutriCheck

מדריך צעד אחר צעד לאיש ה-IT

מערכת ניהול תזונה לבית חולים
אוגוסט 2026

מאגר הקוד

<https://github.com/tomern0305/DiatitionSystem>

תוכן העניינים

3	הקדמה
4	שלב 1 — העתקת הקוד לבית החולים (Fork)
7	שלב 2 — הקמת Supabase
14	שלב 3 — פריסת השרת ב-Vercel
20	שלב 4 — פריסת הלקוח ב-Vercel
23	שלב 5 — כניסה ראשונה ובדיקת קבלה
26	נספח א — תקלות נפוצות
27	נספח ב — המלצות להמשך
27	סיכום

הקדמה

מסמך זה מלווה אותך בהקמת המערכת מאפס — מהעתקת הקוד ועד למערכת עובדת באוויר. אין צורך בידע קודם ב-Vercel או ב-Supabase. כל שלב מלווה בצילום מסך, והריבועים האדומים מסמנים בדיוק על מה ללחוץ.

זמן משוער: כשעה.

מאגר הקוד של המערכת: <https://github.com/tomern0305/DiatitionSystem>

מה נקים

רכיב	שירות	תפקיד
קוד המקור	GitHub	עותק של המערכת בבעלות בית החולים
בסיס נתונים + תמונות	Supabase	הנתונים ותמונות המוצרים
שרת (Back-end)	Vercel	ה-API של המערכת
לקוח (Front-end)	Vercel	הממשק שהמשתמשים רואים

לפני שמתחילים

- חשבון GitHub — עם ארגון (Organization) של בית החולים
- חשבון Supabase — supabase.com
- חשבון Vercel — vercel.com

שים לב: המלצה חשובה: הירשם ל-Supabase ול-Vercel באמצעות חשבון ה-GitHub של בית החולים, ולא באמצעות חשבון פרטי. זה מה שיאפשר ל-Vercel לגשת לקוד, וזה מה שישאיר את הבעלות על המערכת בידי בית החולים לאורך זמן.

עדכונים אוטומטיים

לאחר ההקמה המערכת מתעדכנת אוטומטית: בכל פעם שמישהו דוחף שינוי קוד (push) לענף הראשי ב-GitHub, Vercel בונה ומעלה את הגרסה החדשה לבד — גם לשרת וגם ללקוח. אין צורך בפעולה ידנית.

שלב 1 — העתקת הקוד לבית החולים (Fork)

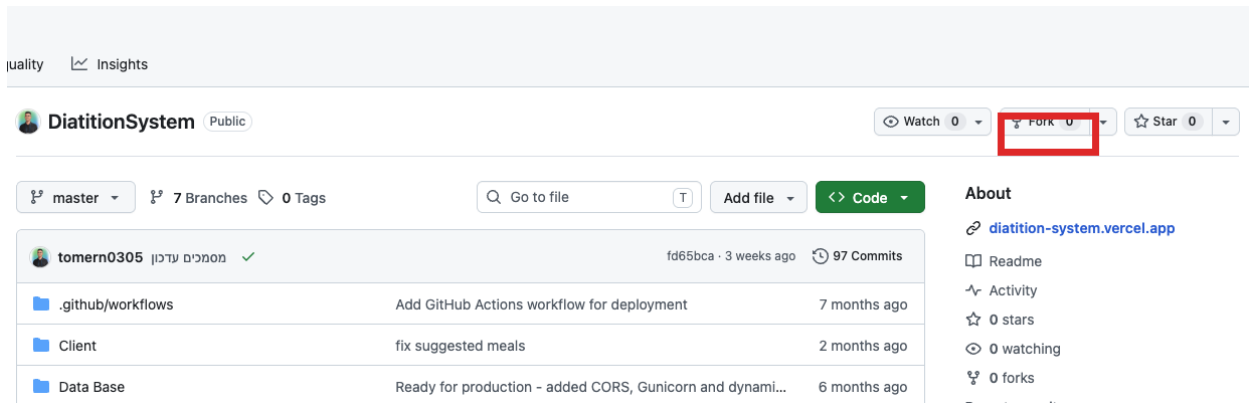
המטרה: ליצור עותק עצמאי של הקוד תחת הארגון של בית החולים. העותק הזה הוא שיתחבר ל-Vercel, והוא זה שבית החולים ישלוט בו.

1.1 פתיחת דף הריפו

היכנס לכתובת מאגר הקוד של המערכת:

<https://github.com/tomern0305/DiatitionSystem>

בפינה הימנית העליונה, מעל רשימת הקבצים, נמצא כפתור Fork. לחץ עליו.



כפתור ה-Fork בדף הריפו

1.2 בחירת היעד

נפתח מסך Create a new fork. בשדה Owner בחר את הארגון של בית החולים. השאר את שם הריפו כפי שהוא ולחץ Create fork.

שים לב: בצילום המסך נבחר חשבון פרטי לצורך ההדגמה, ולכן מופיעה ההודעה "You are creating a fork in your personal account". בהקמה אמיתית יש לבחור ברשימה את הארגון של בית החולים, כדי שהבעלות על הקוד תהיה של בית החולים ולא של אדם מסוים.

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Owner * tomern03051 Repository name * DiatitionSystem

✔ DiatitionSystem is available.

Description

0 / 350 characters

① You are creating a fork in your personal account.

בחירת ה-Owner ולחיצה על *Create fork*

GitHub מעתיק את הקוד. התהליך אורך מספר שניות.

It should only take a few seconds.

מסך ההמתנה

בסיום נפתח דף הריפוי החדש. ודא שהכתובת בשורת הדפדפן מכילה את שם הארגון, שמתחת לשם הריפוי מופיעה השורה `forked from Data Base Client, Server`. ושרשימת התיקיות מכילה את `Data Base Client, Server`.



טיפ: שמור את כתובת הריפוי החדש — תזדקק לה בשלבים 3 ו-4.

שלב 2 — הקמת Supabase

כאן נקים את בסיס הנתונים ואת אחסון התמונות. זהו השלב הארוך ביותר, והוא מחולק לארבעה חלקים.

2.1 יצירת הארגון והפרויקט

היכנס ל-supabase.com והתחבר. אם זו הפעם הראשונה, Supabase יבקש ליצור ארגון (Organization). תן לו שם ובחר את התוכנית Free.

Create a new organization
Organizations are a way to group your projects. Each organization can be configured with different team members and billing settings.

Name

Type Personal

Plan Free - \$0/month

[Cancel](#) [Create organization](#)

יצירת ארגון חדש ב-Supabase

לאחר מכן ייפתח מסך יצירת הפרויקט. מלא את שלושת השדות המסומנים:

שדה	מה למלא
Project name	שם ברור, למשל DiatitionSystem
Database password	לחץ Generate a password — ראה אזהרה למטה
Region	Europe (West EU / Ireland) — הקרוב ביותר לישראל

חשוב: בחר סיסמת בסיס נתונים המורכבת מאותיות וספרות בלבד, בלי תווים מיוחדים כמו # & @ / ? — סיסמה עם סימנים כאלה חייבת קידוד מיוחד בתוך מחרוזת החיבור, וטעות קטנה בקידוד מפילה את השרת בשגיאה שקשה לאבחן. אנחנו נתקלנו בדיוק בזה. סיסמה בת 32 אותיות וספרות חזקה לגמרי ולא דורשת שום טיפול.

שים לב: העתק ושמור את הסיסמה מיד — Supabase לא תציג אותה שוב.

טיפ: בחירת האזור משפיעה ישירות על מהירות המערכת. אזור באירופה נותן זמן תגובה של עשיריות שנייה מישראל; אזור באסיה או בארה"ב מוסיף רבע שנייה לכל שאילתה.

Create a new project
Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization: NutriCheck FREE

GitHub (optional): tomern03051/DiatitionSystem
Ideal for agent-first workflows: update your schema in code, push it to GitHub, and Supabase deploys the changes automatically. [Learn more](#)

Project name: DiatitionSystem

Database password: Copy
This password is strong. [Generate a password](#).

Region: Europe
Select the region closest to your users for the best performance.

Security:

- ☒ **Enable Data API**
Autogenerate a RESTful API for your public schema.
Recommended if using a client library like [supabase-js](#).
- ☒ **Automatically expose new tables**
Grants privileges to Data API roles by default, exposing new tables.
We recommend disabling this to control access manually.
- ☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

ADVANCED CONFIGURATION >

Cancel **Create new project**

שם הפרויקט, הסיסמה והאזור — שלושת השדות המסומנים

2.2 הפעלת התוסף pgvector

התוסף הזה מאפשר לבסיס הנתונים לאחסן וקטורים, ובלעדיו השרת לא יעלה כלל. בתפריט הצד לחץ Database ואז Extensions, חפש vector והפעל את המתג.

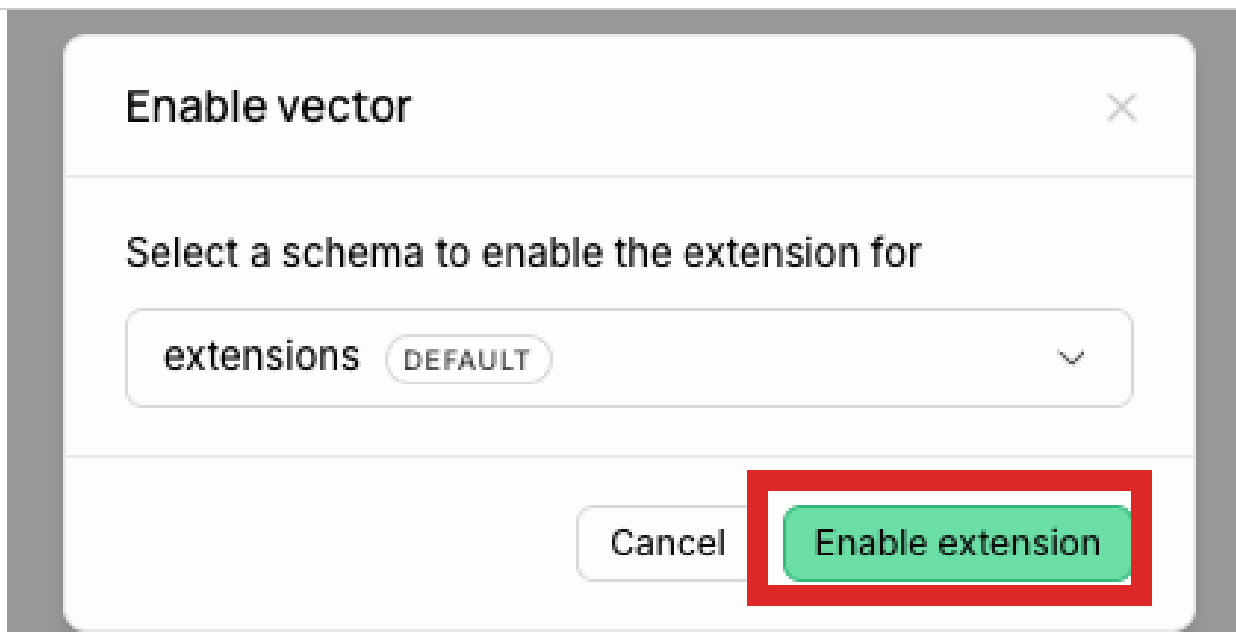
Database Extensions
Manage what extensions are installed in your database Docs

pgvector

NAME	VERSION	SCHEMA	DESCRIPTION	USED BY	LINKS	ENABLED
vector	0.8.2	-	vector data type and ivfflat and hnsw access methods	Supabase Vector Install extension to use Supabase Vector	pgvector/pgvector Docs	☒

חיפוש התוסף vector והמתג להפעלה

יופיע חלון אישור. השאר את הסכימה כברירת המחדל ולחץ Enable extension.



אישור הפעלת התוסף

בסיום המתג יהיה ירוק ובעמודת Schema יופיע extensions.

Database Extensions
Manage what extensions are installed in your database

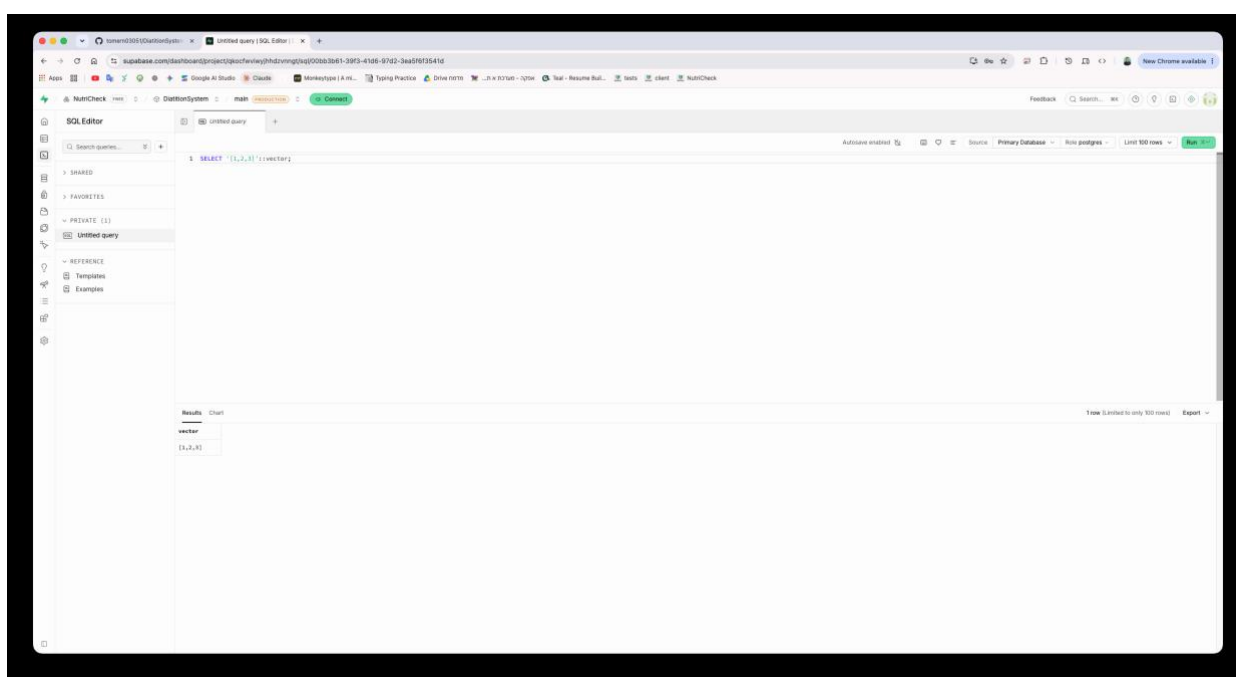
pgvector

NAME	VERSION	SCHEMA	DESCRIPTION	USED BY	LINKS	ENABLED
vector	0.8.2	extensions	vector data type and ivfflat and hnsw access methods	Supabase Vector	pgvector/pgvector Docs	☒

התוסף מופעל

לאימות, פתח את SQL Editor והרץ את השאילתה הבאה. אם חוזרת שורה עם הערך, הכול תקין:

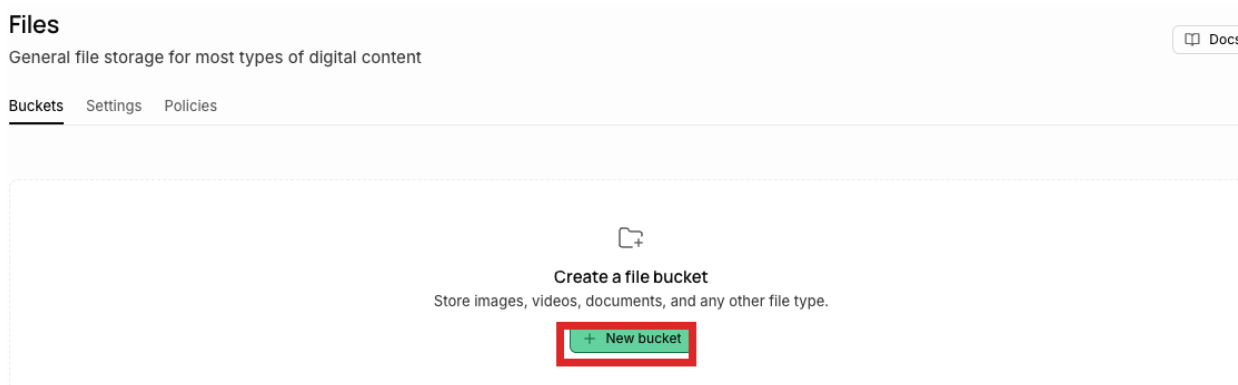
```
SELECT '[1,2,3]':vector;
```



אימות שהטיפוס vector זמין

2.3 יצירת מקום לתמונות המוצרים

בתפריט הצד לחץ Storage ואז New bucket.



כפתור New bucket

בחלון שנפתח מלא את שני הפרטים המסומנים ולחץ Create.

חשוב: שם ה-bucket חייב להיות בדיוק products — באותיות קטנות. השם כתוב בתוך קוד המערכת ואי אפשר לשנותו. בנוסף, חובה להפעיל את המתג Public bucket, אחרת כל תמונות המוצרים יופיעו שבורות באפליקציה.

שם ה-bucket, המתג Public ולחצן היצירה

בסיום ה-bucket יופיע ברשימה עם התג PUBLIC.

Files

General file storage for most types of digital content

Docs

Buckets Settings Policies

Q Search for a bucket	Sorted by created at	Refresh	New bucket
NAME	POLICIES	FILE SIZE LIMIT	ALLOWED MIME TYPES
products PUBLIC	0	Unset (50 MB)	Any

ה-bucket נוצר ומסומן כ-PUBLIC

2.4 איסוף פרטי החיבור

פתח קובץ טקסט זמני ואסוף לשם שלושה ערכים שנצטרך בשלב הבא.

ראשית, מחרוזת החיבור. לחץ על הכפתור הירוק Connect בראש המסך, בחר בלשונית Direct ואז באפשרות Transaction pooler — זו שהפורט שלה 6543.

שים לב: חשוב לבחור דווקא ב-Transaction pooler ולא ב-Direct connection. פונקציות Vercel הן קצרות-חיים ופותרות המון חיבורים; ה-pooler מרבב אלפי לקוחות על מספר קטן של חיבורים אמיתיים, וללא זה בסיס הנתונים ייחנק.

Connect to your project
Choose how you want to use Supabase

Framework
Use a client library

Server
Build APIs

Direct
Connection string

ORM
Third-party library

MCP
Connect your agent

Connection Method

☐ Direct connection
Ideal for applications with persistent and long-lived connections such as those running on virtual machines or long-standing containers.

☒ **Transaction pooler**
Ideal for stateless applications like serverless functions where each interaction with Postgres is brief and isolated.

☐ Session pooler
Only recommended as an alternative to direct connection when connecting via an IPv4 network.

Type URI

Follow these steps Copy prompt

Transaction pooler uses IPv6 by default
Enable the dedicated IPv4 address add-on to connect from IPv4-only networks. [Learn more](#) Enable IPv4 add-on

1 Connection string
Copy the connection details for your database.

Shared pooler Reset database password

```
postgresql://postgres.qkocfwviwjhhdzvnngt:[YOUR-PASSW
```

If your database password contains special characters, [percent-encode](#) them in the connection string.

Connection parameters Copy all

host: aws-1-eu-west-1.pooler.supabase.com

port: 6543

database: postgres

user: postgres.qkocfwviwjhhdzvnngt

בחירת Transaction pooler — שימו לב לפורט 6543

העתק את המחרוזת והחלף בה את [YOUR-PASSWORD] בסיסמה ששמרת (כולל מחיקת הסוגריים המרובעים). התוצאה תיראה כך:

```
postgresql://postgres.<project-ref>:<הסיסמה>@aws-1-eu-west-1.pooler.supabase.com:6543/postgres
```

שנית, כתובת הפרויקט — מופיעה במסך הבית של הפרויקט ונראית כך: `https://<project-ref>.supabase.co` שלישית, מפתח הגישה. לך ל-Project Settings ואז API Keys, והעתק את המפתח שתחת Secret keys (מתחיל ב-sb_secret-).

API Keys

Configure API keys to securely control access to your project

Docs

Publishable and secret API keys

Legacy anon, service_role API keys

Your new API keys are here

We've updated our API keys to better support your application needs. [Join the discussion on GitHub](#)

Having trouble with the new API keys? [Contact support](#)

Publishable key

This key is safe to use in a browser if you have enabled Row Level Security (RLS) for your tables and configured policies.

+ New publishable key

NAME	API KEY
default No description	sb_publishable_nC8Wt0ZAXACnFTqMctavWQ_N1LFH... 
Publishable keys can be safely shared publicly	

Secret keys

These API keys allow privileged access to your project's APIs. Use in servers, functions, workers or other backend components of your application.

+ New secret key

NAME	API KEY
default No description	sb_secret_yokfI.....  

מפתח ה-Secret key — זה שהשרת צריך

חשוב: מפתח ה-Secret עוקף את כל מדיניות ההרשאות של בסיס הנתונים. הוא מיועד לשרת בלבד — לעולם אל תכניס אותו ללקוח, לריפוי, או לכל מקום שנחשף לדפדפן.

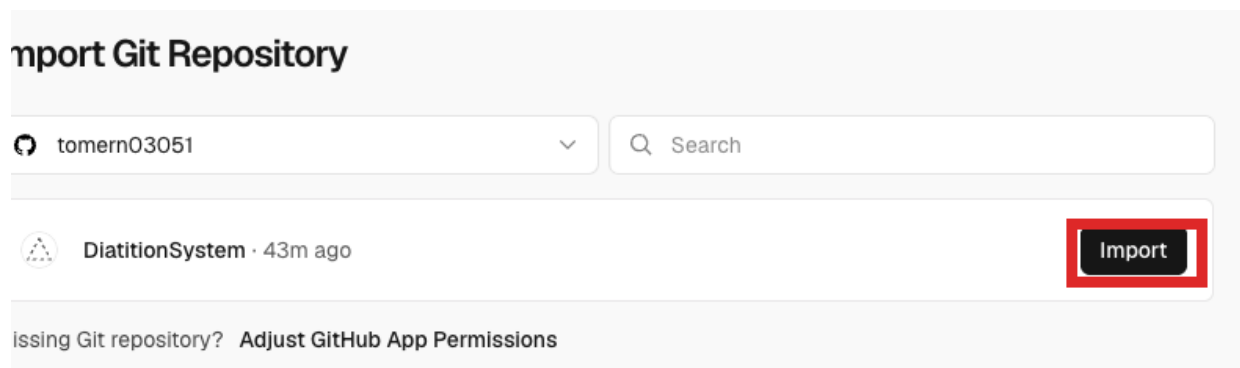
שלב 3 — פריסת השרת ב-Vercel

נקים את הפרויקט הראשון מתוך שניים. שים לב: הלקוח והשרת הם שני פרויקטים נפרדים ב-Vercel, שניהם מאותו ריפוי.

3.1 ייבוא הריפוי

היכנס ל-vercel.com והתחבר עם חשבון ה-GitHub. בדשבורד לחץ Add New ואז Project. אתר את DiatitionSystem ברשימה ולחץ Import.

טיפ: אם הריפוי לא מופיע ברשימה, לחץ על Adjust GitHub App Permissions ואשר ל-Vercel גישה אליו.



ייבוא הריפוי

3.2 בחירת תיקיית המקור

במסך ההגדרות, מצא את השדה Root Directory ולחץ Edit לידו. בחר את התיקייה Server ולחץ Continue.

חשוב: זו הטעות הנפוצה ביותר בשלב הזה. בלי לשנות את Root Directory ל-Vercel Server, ינסה לבנות את כל הריפוי והבנייה תיכשל.

Root Directory

Select the directory containing your source code. For monorepos, create a separate project for each directory you want to deploy.

DiatitionSystem

>	☐ Data Base	
>	☐ Docs	
>	☐ MachineLearning	
>	☐ POC-NaturalLanguageSearch	
>	☐ Prototype	
▼	☒ Server	

Cancel
Continue

בחירת התיקייה Server

לאחר הבחירה, Vercel יזהה אוטומטית את Flask ושדה ה-Root Directory יציג Server.

New Project

Importing from GitHub

 tomern03051/DiatitionSystem  master  Server

Choose where you want to create the project and give it a name.

Ferrel Team

 nutrheck

Hobby

Project Name

diatition-system

Application Preset

 Flask

Root Directory

server

Edit

> Build and Output Settings

> Environment Variables

Deploy

Root Directory מוגדר ל-Server, ו-Flask זוהה אוטומטית

3.3 שם הפרויקט

שנה את Project Name לשם שמבחין בין השרת ללקוח, למשל nutrheck-server. את הלקוח נקרא בהמשך nutrheck-client.

New Project

Importing from GitHub

 tomern03051/DiatitionSystem  master  Server

Choose where you want to create the project and give it a name.

Vercel Team:  nutrheck Hobby 

Project Name: nutrheck-server

Application Preset:  Flask 

Root Directory: Server Edit

 Build and Output Settings

 Environment Variables

Deploy

מתן שם ברור לפרויקט השרת

3.4 משתני הסביבה

פתח את הקטע Environment Variables והוסף את המשתנים הבאים. Vercel יודע לקלוט הדבקה של בלוק שלם לתוך שדה ה-Key הראשון ולפצל אותו לבד.

שם המשתנה	הערך
DATABASE_URL	מחרוזת ה-Transaction pooler מסעיף 2.4
SUPABASE_URL	כתובת הפרויקט ב-Supabase
SUPABASE_SERVICE_KEY	המפתח שמתחיל ב-sb_secret
JWT_SECRET	מחרוזת אקראית — ראו הסבר למטה
AI_ENABLED	false

אין רווחים לפני או אחרי סימן ה-=. את המשתנה AI_ENABLED אפשר להעביר ל-true בהמשך, בתוספת מפתח OpenAI, כדי להפעיל את החיפוש בשפה חופשית.

לייצור המחרוזת האקראית עבור JWT_SECRET, הרץ בטרמינל:

```
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

חשוב: JWT_SECRET הוא המפתח שבו השרת חותם על אסימוני ההתחברות. אם המשתנה אינו מוגדר, הקוד נופל לערך ברירת מחדל קבוע שכתוב בקוד המקור הפומבי — ומי שקורא את הריפו יכול לחתום אסימון משלו ולהיכנס כמנהל, בלי לפרוץ דבר. הגדרתו אורכת דקה ומומלצת בחום לפני חשיפת המערכת לרשת.

> Build and Output Settings

Environment Variables

Key

DATABASE_URL

Value

.....

Environments

Production and Preview

Key

SUPABASE_URL

Value

.....

Environments

Production and Preview

Key

SUPABASE_SERVICE_KEY

Value

.....

Environments

Production and Preview

Key

AI_ENABLED

Value

.....

Environments

Production and Preview

Import .env or paste the .env contents [Learn more](#) + Add More

הזנת משתני הסביבה

כשכל המשתנים ברשימה — לחץ Deploy.

Congratulations!

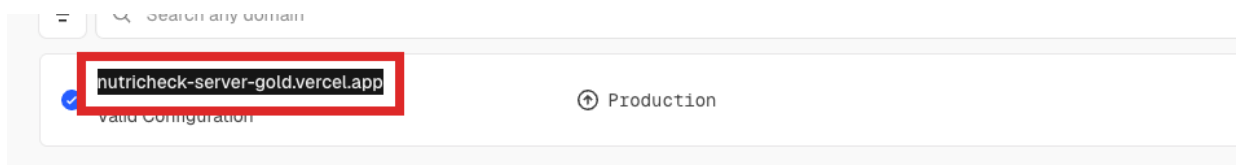
You just deployed a new project to  **nutricheck**.

הפריסה הושלמה

3.5 מציאת כתובת הייצור

לך אל Settings ואז Domains. הכתובת המסומנת כ-Production, זו שאין בה מזהה אקראי, היא הכתובת היציבה של השרת.

חשוב: אל תשתמש בכתובת שמכילה מזהה אקראי (למשל nutricheck-server-2468giadu). היא מצביעה על פריסה אחת ספציפית ותישאר תקועה על הקוד הישן בפריסה הבאה. השתמש תמיד בכתובת ה-Production.



כתובת ה-Production של השרת

3.6 בדיקה

פתח בדפדפן את כתובת הייצור בתוספת /api/status/. התשובה המצופה:

```
{"status": "Flask is running and connected to PostgreSQL!"}
```

זו בדיקה חשובה: היא מוכיחה גם שהשרת רץ וגם שהוא מצליח לדבר עם בסיס הנתונים.



השרת עובד ומחובר לבסיס הנתונים

טיפ: אם קיבלת 404 בכתובת הראשית — זה תקין. השרת מגיש רק נתיבים שמתחילים ב-/api/, ולכתובת הראשית אין דף.

שלב 4 — פריסת הלקוח ב-Vercel

הפרויקט השני, מאותו ריפוי. התהליך זהה לשרת, עם תיקייה אחרת ומשתנה סביבה אחד בלבד. חזור על שלבי הייבוא: Add New, ואז Project, ואז DiatitionSystem, ואז Import. הפעם בחר בתיקייה Client.

Root Directory

Select the directory containing your source code. For monorepos, create a separate project for each directory you want to deploy.

DiatitionSystem

☐ DiatitionSystem (root)

☒ Client

☐ public

☐ src

☐ Data Base

☐ Docs

בחירת התיקייה Client

קרא לפרויקט Vercel. nutricheck-client יזהה אוטומטית את Vite. הוסף משתנה סביבה אחד בלבד:

שם המשתנה	הערך
VITE_API_URL	כתובת ה-Production של השרת, בלי סלאש בסוף

New Project

Importing from GitHub

tomern03051/DiatitionSystem master Client

Choose where you want to create the project and give it a name.

Vercel Team

nutricheck

Hobby

Project Name

nutricheck-client

Application Preset

Root Directory

Client

Edit

> Build and Output Settings

Environment Variables

Key

VITE_API_URL

Value

.....

Environments

Production and Preview

Import .env

or paste the .env contents [Learn more](#)

+ Add More

Deploy

הזנת כתובת השרת

חשוב: הערך של VITE_API_URL נצרב לתוך קבצי ה-JavaScript בזמן הבנייה — הוא אינו נקרא בזמן ריצה. אם תשנה אותו בעתיד, חובה לבצע Redeploy. שינוי המשתנה לבדו לא ישפיע על פריסה קיימת.

לחץ Deploy. הבנייה של הלקוח ארוכה מזו של השרת, כי היא כוללת התקנת חבילות מח וקומפילציה.

Congratulations!

You just deployed a new project to  **nutricheck**.



The screenshot shows the NutriCheck login interface. At the top is the NutriCheck logo, which consists of a green heart with a white fork and spoon inside, and the text "NutriCheck" below it. Under the logo is the text "התחבר לחשבוןך". Below this is a section titled "שם משתמש" (Username) with a text input field and a label "יום השתמש" (Use day). Below that is a section titled "סיסמה" (Password) with a text input field and a label "סיסמה" (Password). At the bottom is a blue button with the text "בניסח" (Login).

הלקוח נפרס — מסך הכניסה מוצג בתצוגה המקדימה

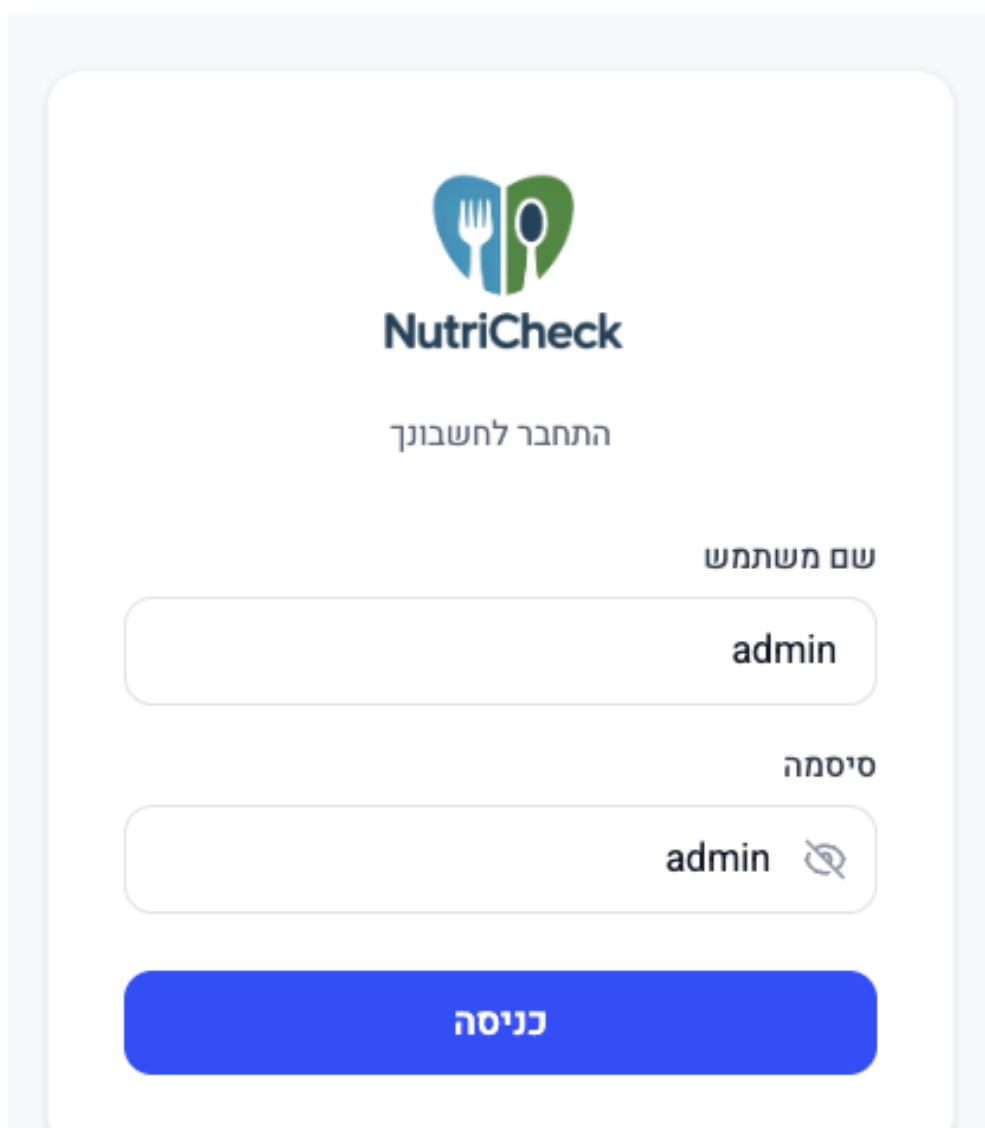
שלב 5 — כניסה ראשונה ובדיקת קבלה

המערכת באוויר. נותר להיכנס, להחליף את סיסמת ברירת המחדל ולוודא שהכול פועל.

5.1 כניסה

פתח את כתובת הלקוח. בהפעלה הראשונה המערכת יוצרת אוטומטית משתמש מנהל:

שדה	ערך
שם משתמש	admin
סיסמה	admin



מסך הכניסה

5.2 החלפת הסיסמה

המערכת תעביר אותך אוטומטית למסך החלפת סיסמה ולא תאפשר להמשיך בלעדיה. בחר סיסמה חזקה ושמור אותה — זו סיסמת המנהל של המערכת.

החלפת סיסמת ברירת המחדל — שלב חובה

טיפ: אם הגעת עד לכאן, השרשרת המלאה מוכתרת: הדפדפן מדבר עם הלקוח, הלקוח עם השרת, והשרת עם בסיס הנתונים — כולל CORS, אסימוני הזדהות ואימות סיסמה.

5.3 בדיקת קבלה

שבע בדיקות שמכסות את כל התשתית:

#	מה לעשות	מה זה מוכיח
1	מסך המוצרים נטען	הלקוח מדבר עם השרת
2	הוסף מוצר חדש עם תמונה	אחסון התמונות ב-Supabase
3	התמונה מוצגת בקטלוג	ה-bucket אכן ציבורי
4	צור ארוחה משני מוצרים	כתיבה מורכבת לבסיס הנתונים
5	נסה למחוק קטגוריה שיש בה מוצרים	המחיקה נחסמת עם הודעה בעברית
6	רענן את הדף בכתובת settings/	ניתוב ה-SPA תקין, לא 404
7	צור משתמש lineworker והתחבר איתו	הרשאות לפי תפקיד

טיפ: בדיקה מספר 2 היא החשובה ביותר — היא הדבר היחיד שבודק בפועל את החיבור ל-Supabase Storage ואת תקינות מפתח ה-Secret.

נספח א — תקלות נפוצות

התקלות הבאות נצפו בפועל במהלך הקמת המערכת.

מה לעשות	הסיבה	התופעה
אפס את סיסמת בסיס הנתונים ב-Supabase לסיסמה מאותיות וספרות בלבד, עדכן את DATABASE_URL ב-Vercel ובצע Redeploy.	הסיסמה במחרוזת החיבור שגויה, או שהיא מכילה תווים מיוחדים שלא קודדו נכון.	password authentication failed "for user" postgres
פתח את Logs בוורסל, אתר את השורה האחרונה של השגיאה, וטפל לפיה.	השרת קרס בעלייה. כמעט תמיד בגלל חיבור כושל לבסיס הנתונים או משתני סביבה חסרים.	500 FUNCTION_INVOCATION_FAILED
הפעל אותו לפי סעיף 2.2.	התוסף pgvector לא הופעל.	type "vector" does not exist
בדוק דרך /api/status.	אין תקלה. השרת מגיש רק נתיבי /api/.	404 בכתובת הראשית של השרת
ודא שהקובץ Client/vercel.json קיים בריפוף.	הגדרת ניתוב ה-SPA חסרה.	404 בעת רענון דף פנימי בלקוח
תקן את הערך ובצע Redeploy.	VITE_API_URL שגוי או שלא בוצע Redeploy אחרי שינויו.	הלקוח נטען אך ריק מנתונים
Storage, ואז products, ואז סמן Public.	ה-bucket אינו ציבורי.	התמונות אינן מוצגות
נפתר מעצמו. אם חוזר בתדירות גבוהה — ראו נספח ב.	שני מופעים של השרת עלו במקביל וניסו ליצור את משתמש המנהל יחד.	duplicate key value בלוגים בעת עלייה

נספח ב — המלצות להמשך

הנקודות הבאות אינן חוסמות את ההקמה, אך מומלץ לטפל בהן לפני שהמערכת נכנסת לשימוש שוטף.

אזור הפונקציה בוורסל

כברירת מחדל השרת רץ בארה"ב בעוד בסיס הנתונים באירלנד, וכל שאילתה חוצה את האוקיינוס פעמיים. ניתן לשנות את האזור בהגדרות הפרויקט לאזור אירופי ולשפר משמעותית את זמני התגובה.

הגבלת CORS

השרת מוגדר כרגע לקבל פניות מכל מקור. לאחר שכתובת הלקוח ידועה, מומלץ להגביל אותו לכתובת זו בלבד.

אימות על נתיבי ה-API

חלק מנתיבי ה-API שמשנים נתונים אינם דורשים אימות. לפני חשיפת המערכת לאינטרנט הפתוח מומלץ להוסיף להם בדיקת הרשאות, או לחסום את השרת ברמת הרשת.

קוד ההפעלה בכל cold start

השרת מריץ פקודות יצירת טבלאות בכל הפעלה קרה של הפונקציה. הדבר מאט את הבקשה הראשונה ומחייב הרשאות DDL קבועות. מומלץ להעביר את הבלוק הזה למתג סביבה שמופעל רק בעת הצורך.

נעילת גרסאות תלויות

קובץ requirements.txt אינו נועל גרסאות. עדכון עתידי של ספרייה עלול לשבור בנייה שעבדה. מומלץ לנעול גרסאות מפורשות.

גיבויים

Supabase מבצעת גיבוי אוטומטי בתוכנית החינמית לטווח מוגבל. לשימוש בייצור מומלץ לוודא מדיניות גיבוי מתאימה.

סיכום

בסיום התהליך קיימים שלושה נכסים בבעלות בית החולים: ריפו ב-GitHub, פרויקט Supabase הכולל את בסיס הנתונים ואת התמונות, ושני פרויקטים ב-Vercel. כל דחיפת קוד לענף הראשי תגרום לעדכון אוטומטי של המערכת. תיעוד טכני מפורט יותר על השרת, הלקוח ובסיס הנתונים נמצא בתיקיית Docs_For_IT בריפו.